

Qualidade de Software

Como o ecossistema Python aborda qualidade via PEPs

Prof. Andre Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Legibilidade e consistência

Documentação

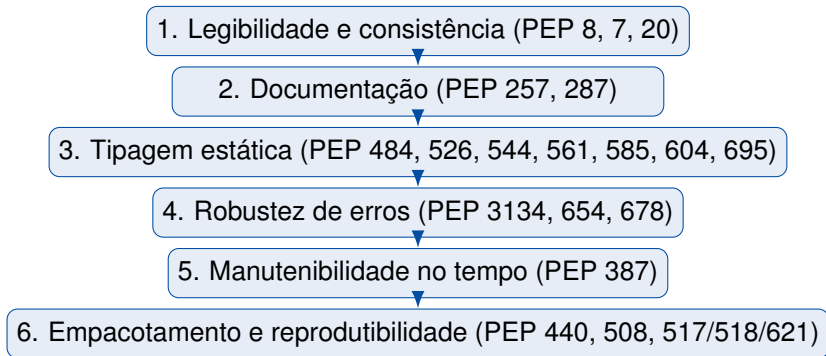
Tipagem estática

Robustez no tratamento de erros

Manutenibilidade no tempo

Empacotamento e reprodutibilidade

Limites e síntese



Dimensões clássicas (ISO/IEC 25010):

- Funcionalidade
- Confiabilidade
- Usabilidade
- Eficiência
- Manutenibilidade
- Portabilidade

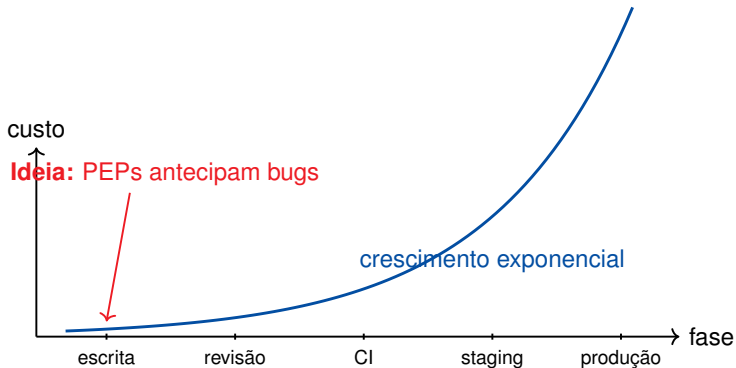
Na prática diária:

- Código legível por humanos
- Bugs capturados cedo
- Falhas diagnosticáveis
- Evolução sem quebrar usuários
- Builds reproduzíveis

Tese da aula

No mundo Python, qualidade não é só “boa prática”—é endereçada **processualmente** via PEPs (Python Enhancement Proposals).

O custo de um bug ao longo do ciclo



Tipagem estática, linters e exceções estruturadas existem para empurrar a detecção *para a esquerda* no eixo.

Python Enhancement Proposal

Documento de desenho que descreve uma nova funcionalidade, processo ou padrão para Python. É o mecanismo **oficial** de evolução da linguagem e do ecossistema.

Tipos

Standards Track (mudanças na linguagem), **Informational** (guias, ex. PEP 8), **Process** (governança, ex. PEP 387).

Legibilidade e consistência

Problema:

Código é lido muito mais vezes do que escrito. Estilo inconsistente cria atrito cognitivo em revisões e onboarding.

Solução:

Convenções universais codificadas em PEPs, automatizadas por formatadores e linters.

PEP 8 — Style Guide for Python Code

Indentação, nomes, espaçamento, comprimento de linha (79/88), imports.

PEP 7 — Style Guide for C Code

Mesmo papel, mas para o código C de CPython.

PEP 20 — The Zen of Python

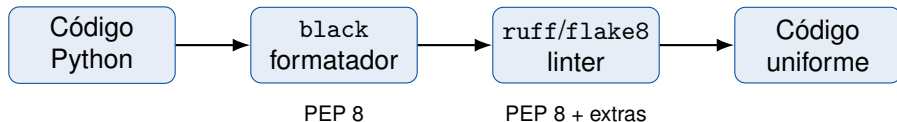
19 aforismos que orientam decisões de design: “*Readability counts*”, “*Simple is better than complex*”, “*Explicit is better than implicit*”.

Antes (não conforme):

```
1  def soma(a,b ):
2      x=a+b
3      return  x
4
5  class minha_classe :
6      def Metodo( self,X ):
7          return X*2
```

Depois (PEP 8):

```
1  def soma(a, b):
2      x = a + b
3      return x
4
5
6  class MinhaClasse:
7      def metodo(self, x):
8          return x * 2
```



`ruff` reimplementa centenas de regras de PEP 8 e outros linters em Rust, sendo ordens de magnitude mais rápido.

O que aprendemos

- PEP 8 e PEP 7 padronizam o visual do código.
- PEP 20 (Zen) é a bússola filosófica.
- `black`, `ruff`, `flake8` *automatizam* a conformidade.



PEP 8



PEP 7



PEP 20

Documentação

Problema:

Documentação externa fica desatualizada. APIs sem descrição forçam leitura do código-fonte para uso básico.

Solução:

Docstrings padronizadas, próximas ao código, extraíveis por ferramentas (Sphinx, pydoc, IDEs).

PEP 257 — Docstring Conventions

Onde colocar docstrings (módulo, classe, função), formato (triplo aspas), estrutura (sumário em uma linha + descrição).

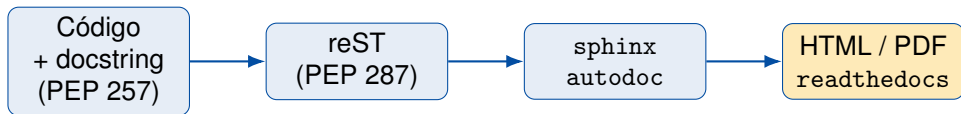
PEP 287 — reStructuredText Docstring Format

Padroniza reST como markup das docstrings; base para Sphinx gerar HTML/PDF.

Estilos populares

Google style, NumPy style e Sphinx/reST — todos compatíveis com PEP 257.

```
1 def normalizar(vetor: list[float]) -> list[float]:
2     """Normaliza um vetor para norma euclidiana 1.
3
4     :param vetor: lista de numeros reais (nao vazia).
5     :returns: novo vetor com mesma direcao e norma 1.
6     :raises ValueError: se ``vetor`` tem norma zero.
7     """
8     norma = sum(x * x for x in vetor) ** 0.5
9     if norma == 0:
10         raise ValueError("vetor nulo nao pode ser normalizado")
11     return [x / norma for x in vetor]
```



O mesmo bloco de docstring alimenta IDEs (autocompletar e *hover*), `help()` no REPL e geração automática de site de documentação — desde que o formato seja consistente.

O que aprendemos

- PEP 257 define *onde* e *como* escrever docstrings.
- PEP 287 padroniza o *markup* (reST), habilitando Sphinx.
- Documentação consistente é insumo para tooling e onboarding.



PEP 257



PEP 287

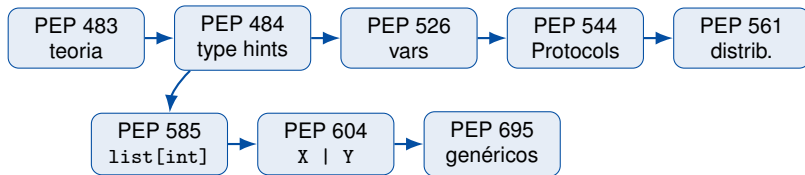
Tipagem estática

Problema:

Erros de tipo em Python “puro” só aparecem em tempo de execução, muitas vezes em produção e longe da causa raiz.

Solução:

Anotações de tipo opcionais + verificadores externos (`mypy`, `pyright`) que capturam erros antes de rodar.



Bases teóricas → sintaxe inicial → extensões e ergonomia.

```
1  # Estilo antigo (PEP 484 + typing)
2  from typing import List, Optional, Union
3
4  def buscar(ids: List[int],
5             filtro: Optional[str] = None) -> Union[str, int, None]:
6      ...
7
8  # Estilo moderno (PEP 585 + PEP 604)
9  def buscar(ids: list[int],
10            filtro: str | None = None) -> str | int | None:
11      ...
```

Genéricos nativos (`list[int]`) e união por barra (`str | None`) eliminam imports e melhoram legibilidade.

```
1  from typing import Protocol
2
3  class TemArea(Protocol):
4      def area(self) -> float: ...
5
6  def total(figuras: list[TemArea]) -> float:
7      return sum(f.area() for f in figuras)
8
9  class Quadrado:                # nao herda de TemArea
10     def __init__(self, l): self.l = l
11     def area(self) -> float: return self.l * self.l
12
13  total([Quadrado(3)])           # ok: compativel estruturalmente
```

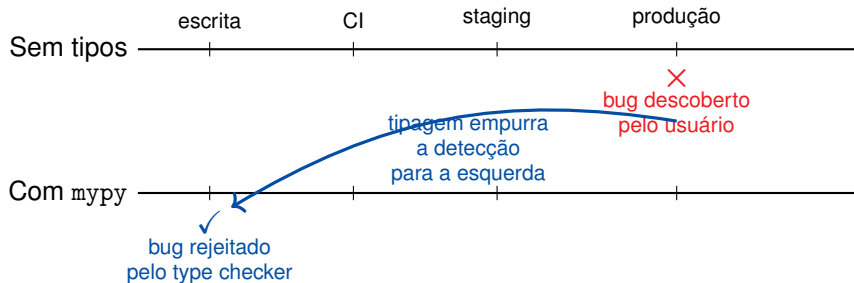
Problema

Como um pacote informa que tem anotações verificáveis por `mypy`?

Solução

Arquivo marcador `py.typed` dentro do pacote + opção declarada no `pyproject.toml`. Sem isso, type checkers ignoram tipos do pacote importado.

Quando o bug é capturado?



Toda anotação x: `int` vira uma asserção verificável *antes* de qualquer execução.

O que aprendemos

- PEPs 483/484 introduziram a fundação teórica e sintática.
- PEPs 526, 544, 585, 604, 695 modernizaram a ergonomia.
- PEP 561 trata a distribuição de tipos em pacotes publicados.
- Habilitam `mypy`, `pyright` e suporte em IDEs.



PEP 483



PEP 484



PEP 526



PEP 544



PEP 561



PEP 585



PEP 604



PEP 695

Robustez no tratamento de erros

Problema:

Tracebacks confusos, causas raiz perdidas, múltiplas falhas concorrentes mascarando umas às outras.

Solução:

Encadeamento, grupos de exceções e notas anexáveis ampliam o diagnóstico sem mudar o fluxo.

```
1  try:
2      parsear(config)
3  except ValueError as e:
4      raise ConfigInvalida("config corrompida") from e
5  # Traceback preserva a ValueError original via __cause__
```

Benefício

Diagnóstico mostra *ambas* as exceções: a causa raiz e a exceção semântica relançada.

```
1  async def coletar():
2      async with asyncio.TaskGroup() as tg:
3          tg.create_task(fetch_a())
4          tg.create_task(fetch_b())
5          tg.create_task(fetch_c())
6      # Se varias tasks falharem, recebe-se um ExceptionGroup
7
8      try:
9          asyncio.run(coletar())
10     except* TimeoutError as eg:
11         log_timeouts(eg.exceptions)
12     except* ValueError as eg:
13         log_validacao(eg.exceptions)
```

```
1  try:
2      processar_linha(linha)
3  except ValueError as e:
4      e.add_note(f"linha {n} do arquivo {caminho}")
5      e.add_note(f"valor original: {linha!r}")
6      raise
```

Anexa contexto *sem* criar exceções novas e sem perder o tipo original— ótimo para frameworks que querem enriquecer erros de terceiros.

O que aprendemos

- PEP 3134: encadeamento preserva causa raiz (`raise ... from`).
- PEP 654: grupos de exceções tratam falhas concorrentes (`except*`).
- PEP 678: notas anexam contexto sem alterar tipo da exceção.



PEP 3134



PEP 654



PEP 678

Manutenibilidade no tempo

Problema:

Mudanças quebrando código de usuários minam confiança e fragmentam o ecossistema.

Solução:

Um processo *previsível* de depreciação com janelas mínimas e avisos claros.

Regras principais

- Mudanças incompatíveis exigem motivação documentada.
- Funcionalidades depreciadas emitem `DeprecationWarning`.
- Janela mínima de **2 releases** entre depreciação e remoção.
- Mudanças silenciosas de comportamento são proibidas.

Por que ler PEP 387?

Modelo de governança replicável em bibliotecas próprias e APIs internas — não só CPython.

O que aprendemos

- Qualidade no longo prazo exige processo, não só código.
- PEP 387 institucionaliza a depreciação progressiva.
- Sirva-se da política para seus próprios pacotes.



PEP 387

Empacotamento e reprodutibilidade

Problema:

“Funciona na minha máquina”:
dependências implícitas, versões
flutuantes, builds não-reprodutíveis.

Solução:

Padrões declarativos para versões,
dependências e builds, todos ancorados
em `pyproject.toml`.

PEP 440 — versionamento

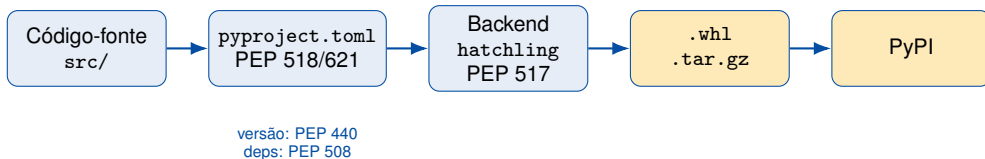
PEP 508 — spec. de dependências

PEP 517 — API de build

PEP 518 — requisitos de build

PEP 621 — metadados em `pyproject.toml`

```
1  [build-system]
2  requires = ["hatchling"]
3  build-backend = "hatchling.build"
4
5  [project]
6  name = "meu-pacote"
7  version = "1.2.0"          # PEP 440
8  requires-python = ">=3.10"
9  dependencies = [
10     "requests>=2.31,<3",    # PEP 508
11     "pydantic~=2.6",
12 ]
13
14 [project.optional-dependencies]
15 dev = ["pytest", "mypy", "ruff"]
```



O mesmo arquivo declarativo (`pyproject.toml`) escolhe o backend, declara dependências e descreve metadados — builds reproduzíveis em qualquer máquina com Python.

O que aprendemos

- PEP 440 e 508 padronizam versões e dependências.
- PEP 517/518 desacoplam builds de `setuptools`.
- PEP 621 centraliza metadados em `pyproject.toml`.
- Resultado: builds previsíveis e auditáveis.



PEP 440



PEP 508



PEP 517



PEP 518



PEP 621

Limites e síntese

Testes automatizados

Apesar de centrais para qualidade, unittest e o ecossistema (pytest, coverage) evoluíram *fora* do processo de PEP.

Implicação prática

Qualidade “oficial” (PEPs) cobre estilo, tipos, erros, pacotes; mas a disciplina de testes vem de *convenções de comunidade* e bibliotecas externas.

Reprodutibilidade — PEP 440, 508, 517, 518, 621

Manutenibilidade — PEP 387

Robustez — PEP 3134, 654, 678

Tipagem — PEP 483, 484, 526, 544, 561, 585, 604, 695

Documentação — PEP 257, 287

Legibilidade — PEP 7, 8, 20

Qualidade emerge da composição das camadas, não de uma única ferramenta.

Setup mínimo (30 min):

- Crie `pyproject.toml` (PEP 621)
- Instale `ruff` + `black`
- Instale `mypy` ou `pyright`
- Adicione `py.typed` (PEP 561)

Hábitos diários:

- Docstring em toda função pública
- Anote tipos nos parâmetros e retornos
- Use `raise X from` e em catches
- Rode `ruff check` no pre-commit

Maturidade crescente

Comece pequeno: 1 PEP por sprint. Em 6 meses sua base de código terá estilo uniforme, tipos verificados e builds reprodutíveis — sem grande refatoração.

Qualidade não é um estado — é um processo.

As PEPs codificam esse processo de forma pública, versionada e auditável.

Dúvidas? andreyoshiaki@dcomp.ufs.br

